Designing a Rate Limiter for PubSub: A Complete Derivation Log

> **Source:** A system design question from Coupang Senior SWE interview reports (Glassdoor): *design a rate limiter for PubSub*.
>
> **Prompt:** A PubSub system has many publishers sending messages to different topics. Design a rate limiter that prevents any single publisher or topic from overwhelming downstream.

---

1. Clarification: Where Does the Limiter Sit?

The prompt says "prevent overwhelming downstream" without saying who downstream is. The path has three segments, each a candidate:

Limiting point	Protects	Nature
publisher → topic	A topic from a single publisher	Internal backpressure
topic → subscriber	A subscriber from its topic (needed under push)	Internal backpressure
user → subscriber	A subscriber from its users	External quota

Two observations worth voicing:

1. **If subscribers pull, the second limiting point is unnecessary** — consumption rate is governed by the subscriber itself. This is an inherent advantage of pull over push.
2. **The third point differs in kind from the first two.** The first two are internal backpressure, with quotas derived from actual downstream capacity; the third is an external quota whose values come from business policy (billing, abuse prevention).

This document covers the first segment only: **publisher → topic**.

---

2. Three SLOs to Establish First

These are not ceremony; each one determines an architectural choice.

2.1 Decision latency

Every message passes through the limiter for one decision. How long does that decision take?

- **Sub-millisecond required** → the decision must be made in local memory; no remote lookups
- **A few milliseconds acceptable** → a centralised counting service becomes viable

This is the limiter's own performance contract, not "the difference in traffic before and after limiting" (that is an effectiveness metric, and belongs to monitoring).

This design targets sub-millisecond, hence local decisions.

2.2 Precision: hard cap or soft cap

Quota is 10K/s — is a brief excursion to 11K acceptable?

- **Hard cap** → globally consistent counting, either querying a central store on every message (slow) or running a distributed protocol (complex)
- **Soft cap** → local counters with periodic reconciliation; fast, with slight overshoot

The key insight: leaving headroom is equivalent to accepting a soft cap. If downstream truly breaks at 20K/s and the limiter threshold is set at 10K, then briefly exceeding 10K is harmless — the real ceiling is far away. **The existence of that headroom is what makes expensive globally consistent counting unnecessary.**

2.3 Granularity: what key does the rule hang on?

Granularity	Counter count (10K topics × 100K publishers)
Per topic	10 thousand
Per publisher	100 thousand
Per (publisher, topic) pair	Up to 1 billion in theory

Choose the pair. It identifies *which* publisher is overwhelming *which* topic rather than applying a blunt instrument.

The billion is a theoretical upper bound. In practice the space is extremely sparse — most publishers send to only a handful of topics, so active pairs may number in the hundreds of thousands. Engineering consequences:

- Create counters **on demand**, never pre-allocate
- Attach a **TTL** so inactive pairs are reclaimed
- Quotas are also on demand — the vast majority use defaults; only a few need explicit configuration

---

3. Scale Assumptions

Item	Value
Topics	10,000
Publishers	100,000
Topic-tier cluster	500 machines
Per-machine capacity	20,000 msg/s
Cluster ceiling	10,000,000 msg/s
Default per-topic quota	10,000 msg/s (large topics up to 500,000)
Peak total traffic	5,000,000 msg/s

---

4. Deployment: Limiter Co-located with Topic Shard

Large topics (500K/s quota): split across 25 machines at 20K quota each, with the limiter co-located on the same machine as the topic shard.

Small topics: several topics share a machine (say 10 per machine), with one limiter owning all decisions for those topics.

Why small topics are not sharded — see the statistical skew analysis in §6.

> **Side effect of co-location: topics on the same machine need isolation.** One topic bursting can consume the whole machine and starve the other nine. Beyond individual quotas, a cap of the form "no single topic may exceed X% of machine capacity" is required.

Quota allocation is typically hybrid: large customers get contractual quotas, everyone else gets defaults or an even split.

---

5. Algorithm: Sliding Window

The limiter's state is `(pub_id, window data)` — O(1) read and write, in local memory, decision in **under 0.1ms**.

Why not a fixed window

A fixed window needs only `(window start, count)` and is the simplest possible implementation, but it suffers the **boundary burst**

of 2×:

```
Quota 10K/s, windows aligned to whole seconds:
  Last 100ms of second 1: send 10K (fills second 1's quota)
  First 100ms of second 2: window resets, send another 10K
  → 20K messages passed in 200ms, twice the quota
```

The limiter considers both windows compliant, but downstream sees an instantaneous 2× spike.

A sliding window (dual-queue implementation) looks at "the past one second" rather than "this whole second", so no 2× burst can appear in any time window.

It solves observability for free

The sliding window's queue can retain a **10-second buffer** of rejected messages' `pub_id` / `message_id`. At 20K/s × 10s = 200K records, memory cost is negligible while diagnostic value is high. **One data structure serves both rate limiting and observability.**

---

6. Routing: Random vs Hash

Once a large topic is split across 25 machines, how are a publisher's messages routed?

6.1 Costs of each option

Routing	Pair-quota accuracy	Hotspot risk
Hash by pub_id	Exact — a publisher always lands on one machine, decided locally with no cross-machine sync	A heavy publisher overwhelms its assigned machine
Random / client-selected	Inexact — traffic spreads across 25 machines, each seeing 1/25	Naturally distributed, no hotspot

The hash hotspot is concrete: if one publisher accounts for 50% of a topic's traffic, hashing sends all of it to one machine, which then carries 50% against a fair share of 4%.

6.2 Statistical skew under random routing

With random routing, "all limiters see roughly equal load" **holds only at high rates**. By the binomial distribution:

Topic rate	Mean per limiter	Coefficient of variation	P99 deviation from mean
500K/s	20,000	0.69%	**+1.6%**
10K/s	400	4.90%	+11.4%
1K/s	40	15.49%	**+36%**

Large topics are indeed uniform (within 2%); small topics deviate far more in relative terms. A 1K/s topic with statically split quotas may see one limiter running 36% above the mean, causing false rejections.

Hence a clean rule: do not shard small topics — one machine owns them entirely. Only large topics are sharded, and at that scale the skew is negligible anyway.

6.3 Tiered decisions: this design's choice

Take random routing, and resolve the pair-quota inaccuracy with tiered decisions:

Path	Decides	Location	Latency
Fast path	Topic-level quota	Local memory	< 0.1ms
Slow path	Publisher-level abuse	Limiters report to a controller, which pushes ban/throttle directives	Seconds

Rationale: abuse detection never needed millisecond response in the first place. The fast path guarantees throughput and latency; the slow path trades immediacy for accuracy.

---

7. Client-Side Selection and Backoff

7.1 The load map and the thundering herd

Publishers read a table of "which machine is idle" before sending. Two problems:

The table becomes a hotspot — one read per message means 5M reads/s at peak. Solution: clients **fetch it once per minute** and cache locally.

The thundering herd, made worse by caching — every publisher sees the *same* map for a full minute. If machine 7 looks idlest, everyone targets machine 7 for that entire minute, overwhelms it, and stampedes elsewhere the next minute, oscillating indefinitely.

Fix: do not pick the idlest — pick randomly among the several idlest (power of two choices). One line of logic disperses the herd, and the one-minute staleness stops mattering.

7.2 Backoff after rejection

Immediately retrying on another machine causes **retry amplification**: if the topic is globally overloaded, one message tries all 25 machines before giving up, so retry traffic during overload is 25× normal — making the overload worse.

Fix: exponential backoff with a retry cap. Wait 2s, then 4s, give up after three attempts. The server can return `Retry-After`, or an explicit "topic globally overloaded" signal that tells the client to stop retrying altogether.

---

8. Headroom: Why 50% Utilisation

Machine ceiling 20K/s, limiter threshold 10K/s, machine count doubled.

This is not conservatism; it is what queueing theory requires. M/M/1 sojourn time is exponentially distributed, so `T(x)/T_mean = −ln(1−x)`, giving **P99 = 4.6 × mean**; and the mean equals service time / (1−ρ):

Utilisation ρ	P99 / service time
0.5	9.2×
0.7	15×
0.9	**46×**

At 90% utilisation the tail is 46× the service time. **Running at 50% protects against bursts and guards P99 simultaneously** — one decision, two benefits.

---

9. Metrics

Limiters report periodically to a controller, which aggregates:

Metric	Purpose
Rejection rate (by topic, by publisher)	Persistently high means quota or capacity mismatch — a scaling signal
Skew across limiters	A sudden increase means uneven traffic or an emerging hot publisher
Distribution of rejected messages	Concentrated in a few publishers → abuse; spread evenly → insufficient capacity. **These call for entirely different responses**
Aggregate volume across limiters	Alert above threshold

The third metric is the most valuable: it separates "ban this publisher" from "add machines", two conclusions that look identical from the rejection rate alone.

---

10. Design Takeaways

1. **Establish which segment is being limited.** The three segments differ in kind: internal backpressure derives quotas from downstream capacity, external quotas from business policy. If subscribers pull, one segment needs no limiter at all.
2. **Headroom removes the need for consistent counting.** Ceiling 20K, threshold 10K — brief overshoot is harmless. That single judgement eliminates the entire complexity of distributed consistent counters.
3. **Granularity determines counter volume.** The pair granularity is a billion in theory but sparse in practice — on-demand creation plus TTL suffices. Do not be scared off by a theoretical upper bound.
4. **Random routing is uniform only at high rates.** A 1K/s topic can deviate 36%, which is why small topics are not sharded.
5. **When pair quotas cannot be decided accurately, tier the decision.** Fast path decides topic quota locally (sub-millisecond); slow path aggregates offline to detect publisher abuse (seconds) — because abuse detection never needed millisecond latency.
6. **A cached load map worsens the thundering herd.** The fix is not a higher refresh rate but "pick randomly among the idlest few".
7. **Retries must back off and cap attempts**, or overload amplifies retry traffic by the machine count.
8. **50% utilisation is simultaneously burst headroom and a P99 guarantee**, with a queueing-theory basis rather than a guess.